SUMMER INTERNSHIP MINI PROJECT REPORT

ON

BINARY APPROACH TO MATLAB

Submitted by:

Sarthak Tripathi - 2401180039

Siddharth – 2401180051

Course Coordinator:

Dr. Ajit Kumar

Dr. Ruchi Singh

Batch – A16

JAYPEE INSTITUTE OF INFORMATION TECHNOLOGY, NOIDA

(U.P.)

1.Smart Lighting System in MATLAB

Aim

To develop an advanced software-only smart lighting system simulation in MATLAB that:

1. Uses MUX/DEMUX logic for dynamic room selection.
2. Implements a priority encoder for motion detection.
3. Uses comparators for ambient light control.
4. Displays LED ON/OFF statuses with decoder-style logic.
5. Adds visualization of light intensity and LED status.
6. Enhances code modularity via a ternary logic function.

Software Components Used

- MATLAB script: smart_lighting_system.m
- Simulated sensor arrays:
  - Motion sensor input
  - Light intensity input
- Functional blocks:
  - Priority encoder
  - Comparator
  - MUX/DEMUX logic
  - Decoder
  - Bar plot visualization
  - Custom ternary function

Methodology

1. Initialization

rooms = 4
motion = [0 1 1 0]
lightIntensity = [300 100 200 400]
threshold = 250
- rooms: total number of simulated rooms
- motion: 1 = motion detected, 0 = no motion
- lightIntensity: simulated sensor values (in lux)
- threshold: light intensity below which light turns ON

2. Priority Encoder

The code scans motion from Room 1 to Room 4. The first room with motion = 1 is assigned as priorityRoom.

3. MUX Logic

Selects the light value and motion state from the priorityRoom.

4. Comparator

If lightIntensity < threshold and motion is detected, the light is turned ON. Else, light remains OFF.

5. DEMUX + Decoder Logic

LED ON/OFF status is simulated using a LEDs array. Only the priorityRoom LED is set to ON, if conditions are met.

6. Display Logic

Results are printed using a ternary() helper function: status = ternary(LEDs(i) == 1, "ON", "OFF");

7. Visualization

A bar() graph shows light intensity per room, ON/OFF status marked in green (ON) or red (OFF), and a red threshold line at 250 lux.

Code And Output

Code

```
% -------- smart_lighting_system.m (Enhanced Version) --------

% ----------------------- Parameters -----------------------

rooms = 4;

motion = [0 1 1 0];                                    % Motion input (1 = detected)

lightIntensity = [300 100 200 400];                    % Simulated light sensor readings

threshold = 250;

% ----------------------- Priority Encoder -----------------------

priorityRoom = -1; for i =

1:rooms

    if motion(i) == 1

        priorityRoom = i; break

    end

end

% ---------- Handling No Motion ---------- if priorityRoom == -1

    disp("No motion detected in any room. All lights OFF."); LEDs = zeros(1, rooms);

else

    % ----------------------- MUX -----------------------

    selectedLight = lightIntensity(priorityRoom); selectedMotion =

    motion(priorityRoom);
```

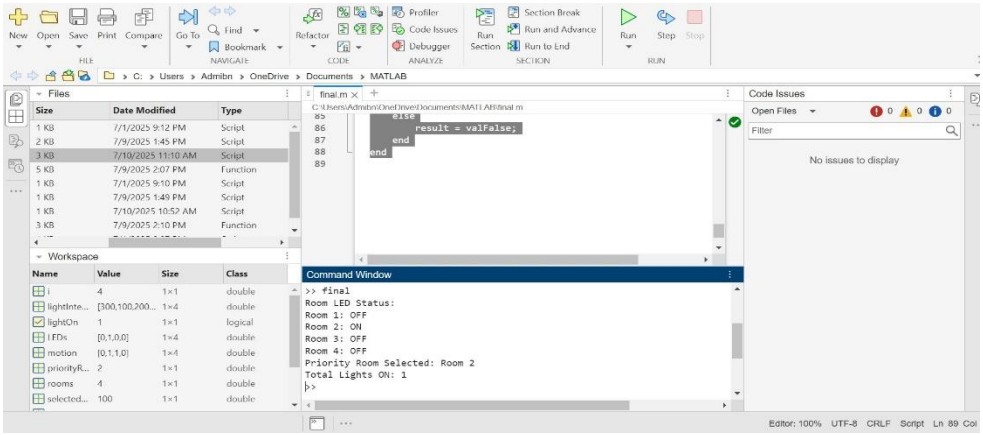
```
    % ---------- Comparator ----------
    if selectedLight < threshold && selectedMotion == 1 lightOn = true;
    else
        lightOn = false;
    end
    % ---------- DEMUX + Decoder ----------
    LEDs = zeros(1, rooms); if lightOn
        LEDs(priorityRoom) = 1;
    end
    % ---------- Display Status ----------
    disp("Room LED Status:"); for i =
    1:rooms
        status = ternary(LEDs(i) == 1, "ON", "OFF"); fprintf("Room %d: %s\n", i,
        status);
    end
    % ---------- Extra Info ----------
    fprintf("Priority Room Selected: Room %d\n", priorityRoom); if ~lightOn
        if selectedLight >= threshold
            disp("Light NOT turned ON due to high ambient light."); elseif selectedMotion == 0
            disp("Light NOT turned ON due to no motion.");
        end
    end
end
% ---------- Summary ----------
```

```
totalLightsOn = sum(LEDs);

fprintf("Total Lights ON: %d\n", totalLightsOn);

% ------------------------  Visualization  ------------------------

figure; bar(lightIntensity); hold on;

for i = 1:rooms if LEDs(i)

    == 1

        text(i, lightIntensity(i)+10, 'ON', 'Color', 'g', ... 'FontWeight','bold','HorizontalAlignment','center');

    else

        text(i, lightIntensity(i)+10, 'OFF', 'Color', 'r', ... 'FontWeight','bold','HorizontalAlignment','center');

    end

end

yline(threshold,'--r','Threshold'); xlabel('Room Number');

ylabel('Light Intensity');

title('Room Light Intensity with ON/OFF Status'); grid on;

% ------------------------  Ternary Function  ------------------------

function result = ternary(cond, valTrue, valFalse) if cond

        result = valTrue; else

        result = valFalse;

    end

end
```

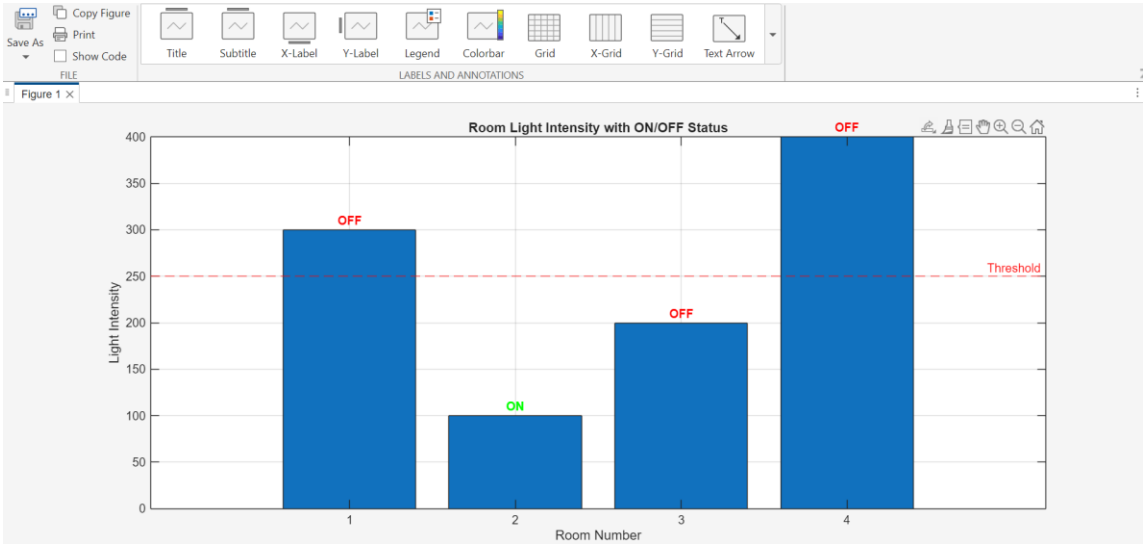
Input:
motion = [0 1 1 0]
lightIntensity = [300 100 200 400]
threshold = 250

Computed:
priorityRoom = 2
lightOn = true
LEDs = [0 1 0 0]

Printed Result:
Room LED Status:
Room 1: OFF
Room 2: ON
Room 3: OFF
Room 4: OFF
Priority Room Selected: Room 2 Total
Lights ON: 1



Graph



Conclusion

This improved simulation demonstrates:

- Real-world embedded logic for smart lighting
- Modular, scalable architecture using software design principles
- A strong foundation for IoT prototype

2. Digital Alarm with Threshold Comparator

Aim

The aim of this mini-project is to design and implement a **Digital Alarm System with Threshold Comparator** in MATLAB, simulating a real-life sensor-based monitoring system. **This project demonstrates the application of digital electronics concepts (comparators, threshold detection, alarm logic) and voice-based human–machine interaction in a simulated environment.**

We monitor simulated sensor readings (randomly generated) and raise alarms when readings cross thresholds.

We also extended the basic idea by adding the following advanced, real-life inspired features:

- Voice-activated start/stop control of the monitoring system.
- Voice-based room selection for monitoring among four rooms.
- Voice commands to choose the type of monitoring output: time-series sensor readings, alarm level counts as a bar graph, or both.
- Continuous, interactive monitoring with the ability to switch rooms or exit on command.

Software Components Used

- **MATLAB :**
  - Sensor signal simulation.
  - Threshold detection using comparators.
  - Visualization of sensor readings and alarm status.
  - MATLAB–Python integration using the py. interface.
- **Python** :
  - Speech recognition using the speech_recognition library.
- **MATLAB-Python Integration**:
  - py.voice_command.listen_command() called from MATLAB to capture voice input.

Methodology

1. Simulate readings from four independent rooms as random time-series data (rand(1,100)*100).

2. Define two thresholds:

- **Warning threshold** (e.g., 50).
- **Critical threshold** (e.g., 70).

3. Compare each sensor reading with thresholds using digital comparator logic:

- If > critical : mark as Critical Alarm.
- If > warning but ≤ critical : mark as Warning.
- Otherwise : Normal.

4. Implement continuous monitoring loop:

- Start/stop controlled via voice commands (start / stop).
- Room selection controlled via voice (Room 1, Room 2, etc.).
- Output selection controlled via voice (sensor readings, bar graph, or everything).

5. Display results visually:

- Time-series plot of sensor data with markers for critical/warning points.
- Bar graph summarizing counts of Normal, Warning, and Critical alarms.

6. Return to start and allow the user to monitor another room or exit the system.

Code and Output

Matlab Code

```
clc; clear; close all;

%% Initial Setup
n_rooms = 4;
n_samples = 100;
data = rand(n_rooms, n_samples) * 100;
threshold_warning = 50;
threshold_critical = 70;

disp('Voice-Activated Digital Alarm System');

while true
    %% Start/Stop Prompt
    disp('Say "start" to begin monitoring, or "stop" to exit.');
    cmd = string(py.voice_command.listen_command());
    disp("You said: " + cmd);

    if strcmpi(cmd, 'stop') || strcmpi(cmd, 'end')
        disp('Exiting program...');
        break;
    elseif ~strcmpi(cmd, 'start')
        disp('Unrecognized command. Please say "start" or "stop".');
        continue;
    end

    %% Room Selection
    disp('Available rooms: Room 1, Room 2, Room 3, Room 4');
    disp('Say which room to monitor (e.g., "Room 2")');

    while true
        cmd = string(py.voice_command.listen_command());
        disp("You said: " + cmd);

        if strcmpi(cmd, "room 1")
            room = 1; break;
        elseif strcmpi(cmd, "room 2")
            room = 2; break;
        elseif strcmpi(cmd, "room 3")
            room = 3; break;
        elseif strcmpi(cmd, "room 4")
            room = 4; break;
        else
            disp('Please say: "Room 1", "Room 2", "Room 3", or "Room 4"');
        end
    end
```

```
    disp(['Monitoring Room ', num2str(room)]);

    %% Process Room Data
    room_data = data(room, :);

    alarm_level = strings(1, n_samples);
    for i = 1:n_samples
        if room_data(i) > threshold_critical
            alarm_level(i) = "Critical";
        elseif room_data(i) > threshold_warning
            alarm_level(i) = "Warning";
        else
            alarm_level(i) = "Normal";
        end
    end

    n_critical = sum(alarm_level == "Critical");
    n_warning  = sum(alarm_level == "Warning");
    n_normal   = sum(alarm_level == "Normal");

    %% What to view
    disp('What would you like to see?');
    disp('Say: "sensor readings", "bar graph", or "everything"');

    while true
        cmd = string(py.voice_command.listen_command());
        disp("You said: " + cmd);

        if any(strcmpi(cmd, {'sensor readings', 'bar graph', 'everything'}))
            break;
        else
            disp('Please say: "sensor readings", "bar graph", or "everything"');
        end
    end

    %% Show outputs
    if strcmpi(cmd, 'sensor readings') || strcmpi(cmd, 'everything')
        figure;
        plot(room_data, '-o', 'MarkerSize', 4, 'DisplayName', 'Sensor Data');
        hold on;
        plot(find(alarm_level=="Critical"), room_data(alarm_level=="Critical"), 'rx', 'MarkerSize', 8, 'LineWidth', 1.5, 'DisplayName', 'Critical');
        plot(find(alarm_level=="Warning"), room_data(alarm_level=="Warning"), 'gx', 'MarkerSize', 8, 'LineWidth', 1.5, 'DisplayName', 'Warning');
        yline(threshold_warning, '--g', 'Warning Threshold');
        yline(threshold_critical, '--r', 'Critical Threshold');
        xlabel('Time Sample');
        ylabel('Sensor Reading');
```

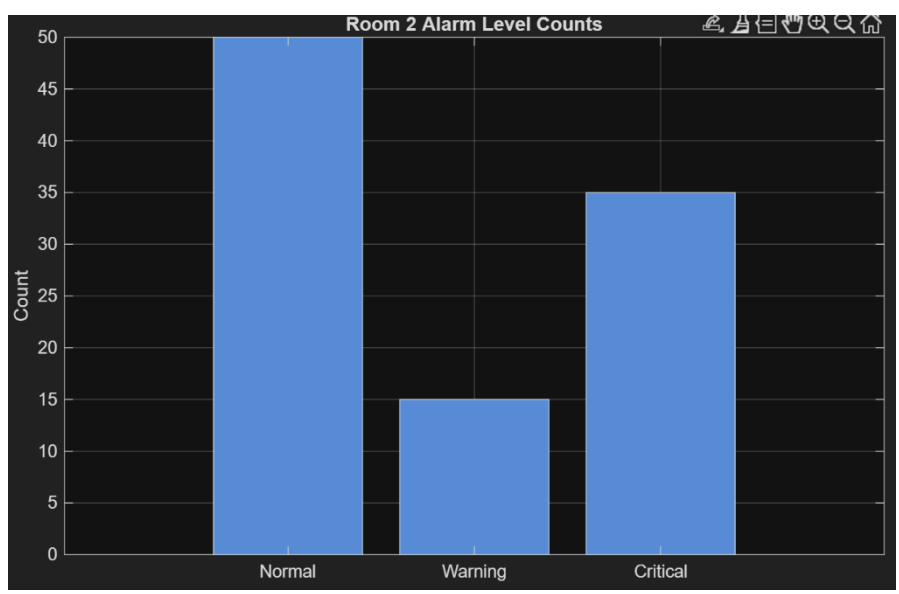
```
        ylabel('Sensor Reading');
        title(sprintf('Room %d Sensor Readings with Alarms', room));
        legend();
        grid on;
    end

    if strcmpi(cmd, 'bar graph') || strcmpi(cmd, 'everything')
        figure;
        bar([n_normal n_warning n_critical],'FaceColor','flat');
        set(gca,'XTickLabel',{'Normal','Warning','Critical'});
        ylabel('Count');
        title(sprintf('Room %d Alarm Level Counts', room));
        grid on;
    end

    %% End of session message
    disp('Session complete.');
    disp('Say "start" to monitor another room, or "stop" to exit.');
end

disp('Program exited.');
```

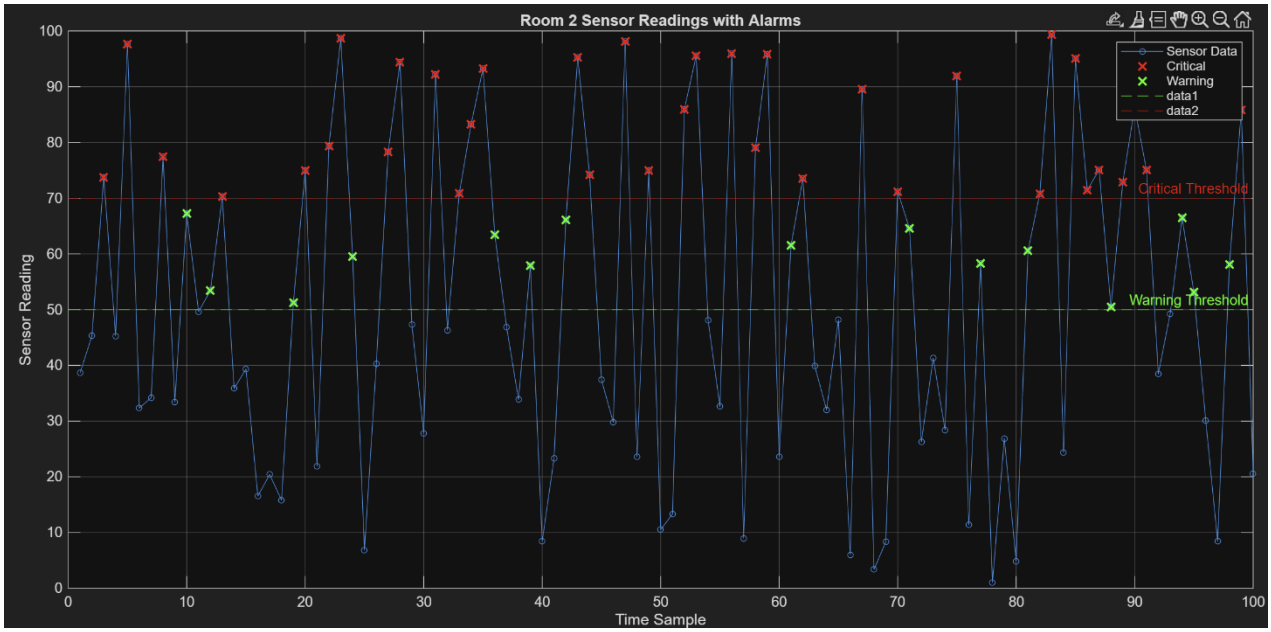
Python Code

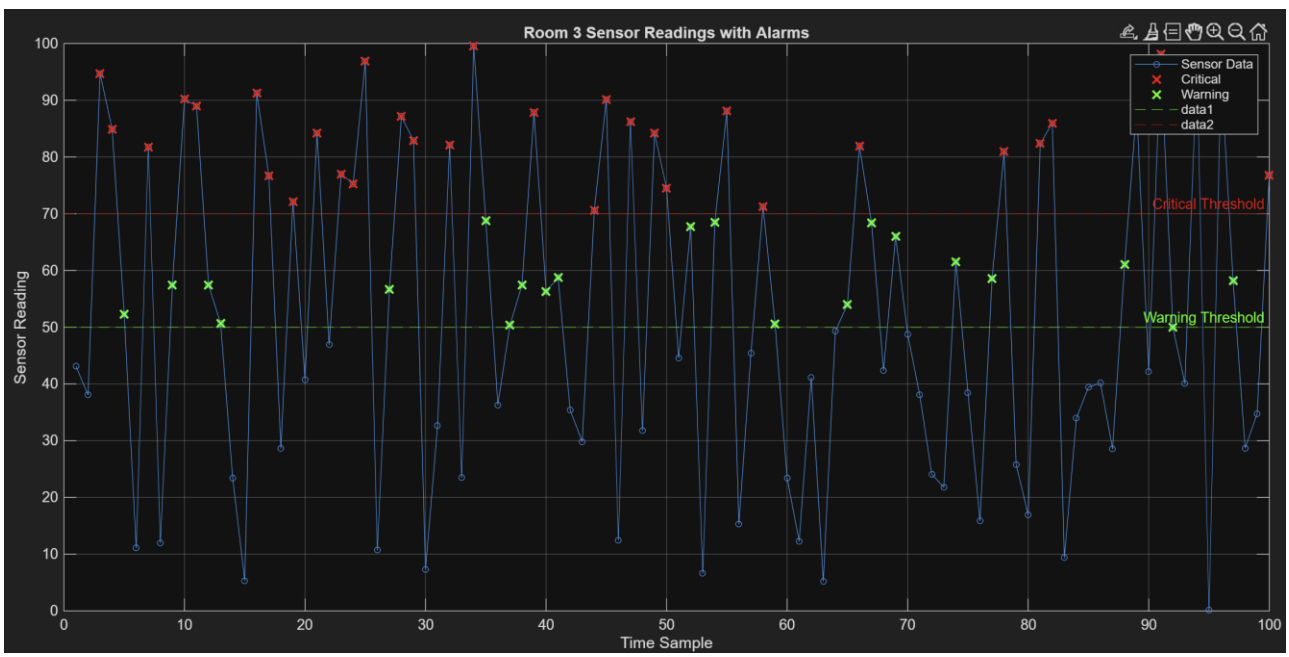
```
import speech_recognition as sr

def listen_command(timeout=30):
    r = sr.Recognizer()
    with sr.Microphone() as source:
        print(f"Listening for up to {timeout} seconds...")
        audio = r.listen(source, timeout=timeout)
    try:
        cmd = r.recognize_google(audio)
        print(f"Recognized: {cmd}")
        return cmd
    except:
        return "unrecognized"
```

Output

```
Voice-Activated Digital Alarm System
Say "start" to begin monitoring, or "stop" to exit.
Listening for up to 30 seconds...
Recognized: start
You said: start
Available rooms: Room 1, Room 2, Room 3, Room 4
Say which room to monitor (e.g., "Room 2")
Listening for up to 30 seconds...
Recognized: room 3
You said: room 3
Monitoring Room 3
What would you like to see?
Say: "sensor readings", "bar graph", or "everything"
Listening for up to 30 seconds...
Recognized: sensor readings
You said: sensor readings
Session complete.
Say "start" to monitor another room, or "stop" to exit.
Say "start" to begin monitoring, or "stop" to exit.
Listening for up to 30 seconds...
Recognized: start
You said: start
Available rooms: Room 1, Room 2, Room 3, Room 4
Say which room to monitor (e.g., "Room 2")
Listening for up to 30 seconds...
Recognized: Room 2
You said: Room 2
Monitoring Room 2
What would you like to see?
Say: "sensor readings", "bar graph", or "everything"
Listening for up to 30 seconds...
Recognized: everything
```




Room 2 Sensor Readings with Alarms
Sensor Data
Critical
Warning
data1
data2
Critical Threshold
Warning Threshold
Sensor Reading
Time Sample
0
10
20
30
40
50
60
70
80
90
100


Room 3 Sensor Readings with Alarms
Sensor Data
Critical
Warning
data1
data2
Critical Threshold
Warning Threshold
Sensor Reading
Time Sample
0
10
20
30
40
50
60
70
80
90
100

Results

- Successfully simulated a digital alarm system monitoring four rooms.
- Detected and classified alarms as Critical, Warning, or Normal based on comparator logic.
- Displayed the results in a clear, visual manner.
- Integrated voice control for a more natural, hands-free user experience.
- Achieved continuous, real-time interaction allowing multiple rooms to be monitored in one session.

Conclusion

This project demonstrated a practical implementation of **digital comparator-based alarm logic using MATLAB**, enhanced with modern voice-controlled human–machine interaction.

By simulating sensor data and applying threshold logic, we replicated a real-world monitoring system that could be used in industries, smart buildings, or laboratories.

The integration of Python speech recognition with MATLAB provided an intuitive and user-friendly interface, showcasing the potential of combining traditional digital electronics concepts with advanced technologies.